
Oliver Krauss

State Machines & Game AI

ALD | MTD.ba VZ SS26

Hagenberg 24.04.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Lecture 8 -- AI-Assisted Coding

Last session: AI as a coding copilot

We used AI to generate code faster -- with context, guardrails, and human responsibility.

Recap: Lecture 8 -- AI-Assisted Coding

Last session: AI as a coding copilot

We used AI to generate code faster -- with context, guardrails, and human responsibility.

How we guided AI output:

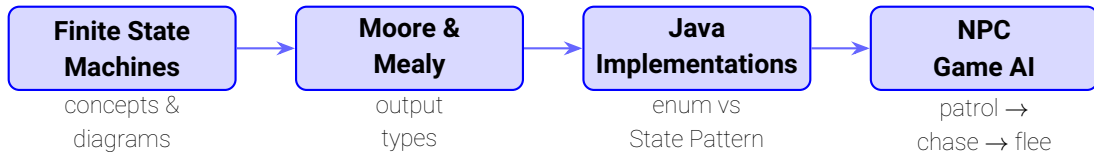
- CLAUDE.md / AGENTS.md project rules
- Spec → Plan → Review → Execute loop
- Feature files for scoping
- Always read the diff

What still stayed on us:

- AI generates plausible, not always correct code
- OWASP Top 10 still applies
- EU AI Act: developer obligations
- You own everything you submit

Today: Let us look at AI differently: not as a coding assistant, but as behaviour in a game. How do we make an NPC patrol, chase, or flee? Finite State Machines give us a clear, testable model.

Today's Journey



What Is a State Machine?

Real-world systems that “remember”:

- A traffic light cycles through colours
- A door is locked or unlocked
- A vending machine waits for coins
- An NPC patrols, chases, attacks, flees

What Is a State Machine?

Real-world systems that “remember”:

- A traffic light cycles through colours
- A door is locked or unlocked
- A vending machine waits for coins
- An NPC patrols, chases, attacks, flees

What they share

At any point in time they are in **exactly one state**.

An **event** causes a **transition** to a new state.

What Is a State Machine?

Real-world systems that “remember”:

- A traffic light cycles through colours
- A door is locked or unlocked
- A vending machine waits for coins
- An NPC patrols, chases, attacks, flees

What they share

At any point in time they are in **exactly one state**.

An **event** causes a **transition** to a new state.

Finite State Machine (FSM)

A formal model of a system with:

- a finite set of **states**
- a set of **inputs / events**
- **transition rules**: given a state and an input, produce the next state
- one **initial state**
- optional **output** per state or transition

FSM Components

Textbook vocabulary. You'll see these symbols in literature. No need to memorize them; focus on what each one means.

S	states	finite set of states
Σ	sigma	input alphabet (events)
δ	delta	transition function : $S \times \Sigma \rightarrow S$
s_0	s-zero	initial state , $s_0 \in S$
λ	lambda	output function (Moore or Mealy)

FSM Components

Textbook vocabulary. You'll see these symbols in literature. No need to memorize them; focus on what each one means.

S	states	finite set of states
Σ	sigma	input alphabet (events)
δ	delta	transition function : $S \times \Sigma \rightarrow S$
s_0	s-zero	initial state , $s_0 \in S$
λ	lambda	output function (Moore or Mealy)

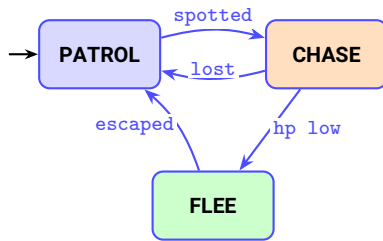
Guards

A **guard** is a condition that must be true for a transition to fire: `[player.distance < 50]`

FSM Components

Textbook vocabulary. You'll see these symbols in literature. No need to memorize them; focus on what each one means.

S	states	finite set of states
Σ	sigma	input alphabet (events)
δ	delta	transition function : $S \times \Sigma \rightarrow S$
s_0	s-zero	initial state , $s_0 \in S$
λ	lambda	output function (Moore or Mealy)



same NPC we'll build today

Guards

A **guard** is a condition that must be true for a transition to fire: `[player.distance < 50]`

State Diagrams -- Notation

Core symbols:

- **Circles** represent states
- **Arrow without source** marks the initial state
- **Labelled arrows** represent transitions (event / guard)
- **Double circle** marks an accepting / final state (if needed)

Reading tip

Focus first on *states*, then follow outgoing transitions by event labels.

Self-loops are valid transitions: the state stays the same, but the event is still handled.

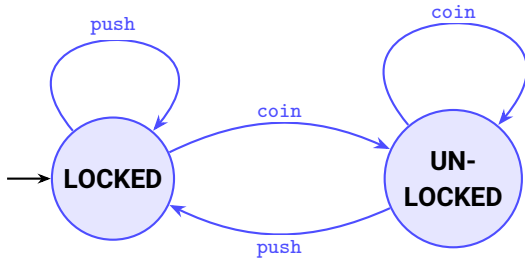
State Diagrams -- Example: Turnstile

Scenario

A subway turnstile has two states:

- **LOCKED**
- **UNLOCKED**

Events: **coin**, **push**



Moore Machines -- Output Belongs to States

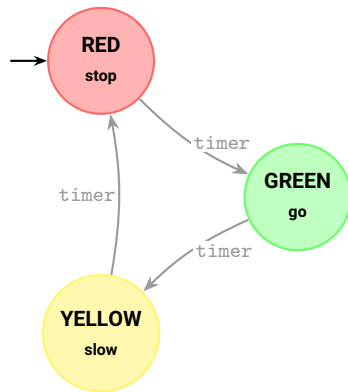
Moore Machine

Output is associated with **states**.

The output depends *only* on the current state -- not on the input that caused the transition.

Characteristics:

- Output changes when state changes
- Easier to reason about ("what is this state *doing*?")
- More states may be needed than Mealy
- **Natural fit for OOP**: each state class *owns* its behaviour



Each state node carries its output (stop/go/slow).

Mealy Machines -- Output Belongs to Transitions

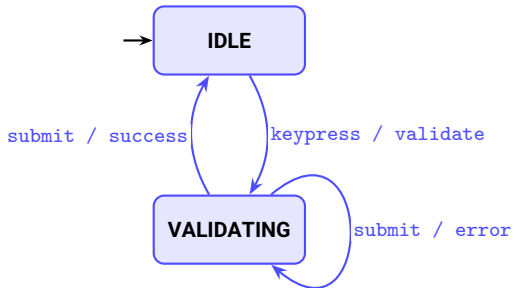
Mealy Machine

Output is associated with **transitions**.

The output depends on *both* the current state and the input event.

Characteristics:

- Output tied to specific event reactions
- Often fewer states than equivalent Moore machine
- Transition labels become **input** / **output**
- Natural fit for **reactive systems**: modems, protocols, UI event handlers



Each arrow carries the output (validate/error/success).

Moore vs. Mealy -- At a Glance

	Moore	Mealy
Output lives in	state node	transition arrow
Output depends on	current state only	state + input
Typical state count	more	fewer
Diagram readability	cleaner nodes	busier arrows
OOP mapping	state class <i>owns</i> behaviour	transition handler
Typical use cases	NPC AI, UI screens	modems, protocols, parsers

Moore vs. Mealy -- At a Glance

	Moore	Mealy
Output lives in	state node	transition arrow
Output depends on	current state only	state + input
Typical state count	more	fewer
Diagram readability	cleaner nodes	busier arrows
OOP mapping	state class <i>owns</i> behaviour	transition handler
Typical use cases	NPC AI, UI screens	modems, protocols, parsers

Key heuristic

Ask: "Is the output about **being** in a state, or about **reacting** to an event?"

Being → Moore. **Reacting** → Mealy.

Example: Traffic Light (Moore)

Traffic light = classic Moore machine.

The colour displayed is the output of the *current state* -- not of a particular transition.

State	Output (display)
RED	stop
GREEN	go
YELLOW	slow down

Example: Traffic Light (Moore)

Traffic light = classic Moore machine.

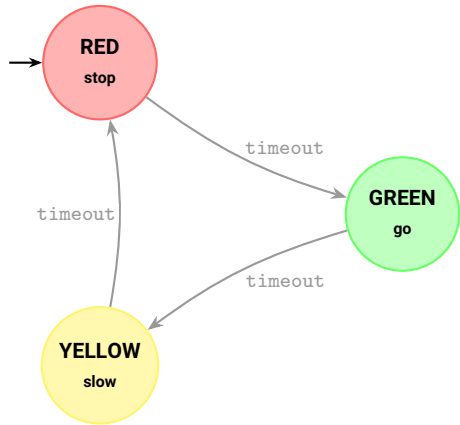
The colour displayed is the output of the *current state* -- not of a particular transition.

State	Output (display)
RED	stop
GREEN	go
YELLOW	slow down

Transition trigger

Each state has a **timer**.

timeout event fires the transition.



Example: Vending Machine (Mealy) I

What to look for:

- In a Mealy machine, **output lives on transitions**
- Labels use **input / output**
- The same state can emit different outputs, depending on the event

One concrete flow

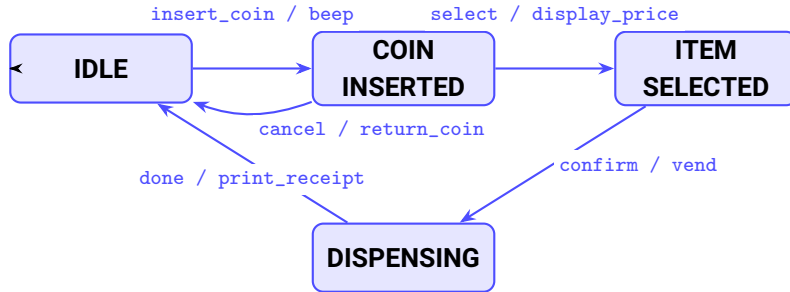
1. `insert_coin / beep`
2. `select / display_price`
3. `confirm / vend`
4. `done / print_receipt`

Reading strategy

Follow one user flow and read each edge as:

event happens / machine reacts immediately

Example: Vending Machine (Mealy) II



Each transition carries **input / output** -- the output (beep, display, vend) is a reaction to the event, not a property of the state. This is the Mealy signature.

Interactive Break #1 -- Moore or Mealy?

For each scenario: decide Moore or Mealy and justify in one sentence.

1. **Elevator**: the doors open while the cabin is at a floor and closed while moving. The floor display shows the current floor.
2. **Login form**: typing a character validates the field; submitting triggers either "success" or "error message".
3. **NPC**: the enemy patrols, chases, attacks, and flees. Each mode has a distinct animation and movement speed.

Hint

Ask yourself: does the output/behaviour describe *what the system is doing right now* (Moore) or *what happens in response to a specific input* (Mealy)?

Interactive Break #1 -- Answer

1 -- Elevator: **Moore**

The door state (open/closed) and floor number are *outputs of the current state*. "At floor 3, doors open" -- the state owns the output.

2 -- Login form: **Mealy**

"Success" and "error" messages are *reactions to the submit event*. The same VALIDATING state produces different outputs depending on whether the submit is valid or invalid.

3 -- NPC: **Moore**

Animation and speed are *properties of being in a state*. The PATROL state always plays the walk cycle -- regardless of how we entered it.

This is what we'll build today.

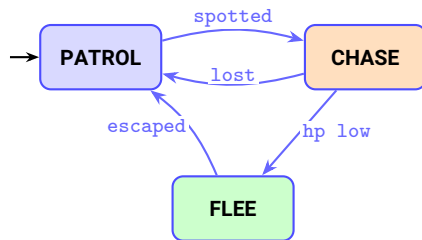
The heuristic in action

Being → Moore | Reacting → Mealy | NPC AI = Moore → State Pattern

FSMs Are Directed Graphs

FSMs are graphs

FSM concept	Graph concept
State	Vertex (node)
Transition	Directed edge
Event label	Edge label / weight
Guard	Edge predicate

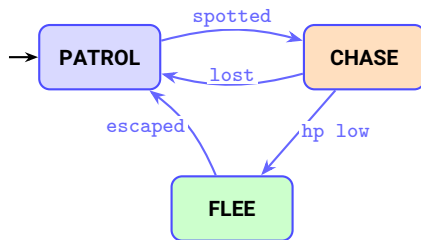


vertices = states, edges = transitions

FSMs Are Directed Graphs

FSMs are graphs

FSM concept	Graph concept
State	Vertex (node)
Transition	Directed edge
Event label	Edge label / weight
Guard	Edge predicate



vertices = states, edges = transitions

Why this matters

Because FSMs are graphs, every graph algorithm applies for free:

Reachability: *can the NPC ever reach FLEE from PATROL?*

Shortest path: *what is the fastest route to ATTACK?*

Cycle detection: *is there a state the NPC can never escape?*

Representing a Graph in Java

Adjacency Matrix

	A	B	C	D
A	0	1	1	0
B	0	0	1	0
C	0	0	0	1
D	1	0	0	0

Pros: instant edge lookup (row + column).

Cons: wastes space when most cells are empty (sparse graphs).

Adjacency List

A $\rightarrow \{B, C\}$

B $\rightarrow \{C\}$

C $\rightarrow \{D\}$

D $\rightarrow \{A\}$

Pros: efficient -- only stores edges that actually exist.

Cons: checking a specific edge requires scanning the list.

Representing a Graph in Java

Adjacency Matrix

	A	B	C	D
A	0	1	1	0
B	0	0	1	0
C	0	0	0	1
D	1	0	0	0

Pros: instant edge lookup (row + column).

Cons: wastes space when most cells are empty (sparse graphs).

Adjacency List

A → {B, C}

B → {C}

C → {D}

D → {A}

Pros: efficient -- only stores edges that actually exist.

Cons: checking a specific edge requires scanning the list.

Adjacency List in Java: **Map<State, List<Transition>>**

This is exactly how we will represent an FSM engine internally. Each key = a state; value = list of outgoing transitions.

From State Diagram to Adjacency List

```
1  // A transition: target state + label (guard logic stays in update())
2  class Transition {
3      NPCState target;
4      String event;
5      Transition(NPCState target, String event) {
6          this.target = target;
7          this.event = event;
8      }
9  }
10 // FSM as adjacency list: each state maps to its outgoing transitions
11 Map<NPCState, List<Transition>> fsm = new HashMap<>();
12
13 fsm.put(NPCState.PATROL, List.of(
14     new Transition(NPCState.CHASE, "player spotted")
15 ));
16 fsm.put(NPCState.CHASE, List.of(
17     new Transition(NPCState.ATTACK, "in range"),
18     new Transition(NPCState.PATROL, "lost player")
19 ));
```

Two Approaches

Approach A -- Enum-based FSM

- States as **enum** constants
- Transitions via **switch** statement in context
- Behaviour as abstract methods on enum
- **Good for**: simple machines, few states, no complex OOP hierarchy
- **Drawback**: switch grows large; hard to extend without modifying it

Approach B -- State Pattern (GoF)

- Each state is a **class** implementing a **State** interface
- Context delegates behaviour to the current state object
- Adding a state = adding a new class, without changing any existing code
- **Good for**: many states, complex behaviour, team projects
- **Drawback**: more files; can be over-engineering for 2-state machines

Both are valid -- choosing between them is a design decision, not a right/wrong question.

Approach A: Enum-Based FSM

```
1  enum OrderStatus {
2      PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED;
3      // Transition logic: given an event, return the next state
4      public OrderStatus next(String event) {
5          return switch (this) {
6              case PENDING      -> event.equals("pay")      ? PROCESSING : this;
7              case PROCESSING   -> event.equals("dispatch")? SHIPPED    : this;
8              case SHIPPED      -> event.equals("arrive")    ? DELIVERED  : this;
9              default           -> this; // terminal states ignore events
10         };
11     }
12 }
13
14 // Usage
15 OrderStatus status = OrderStatus.PENDING;
16 status = status.next("pay"); // -> PROCESSING
```

Clean for small machines. The **switch** is entirely inside the enum.

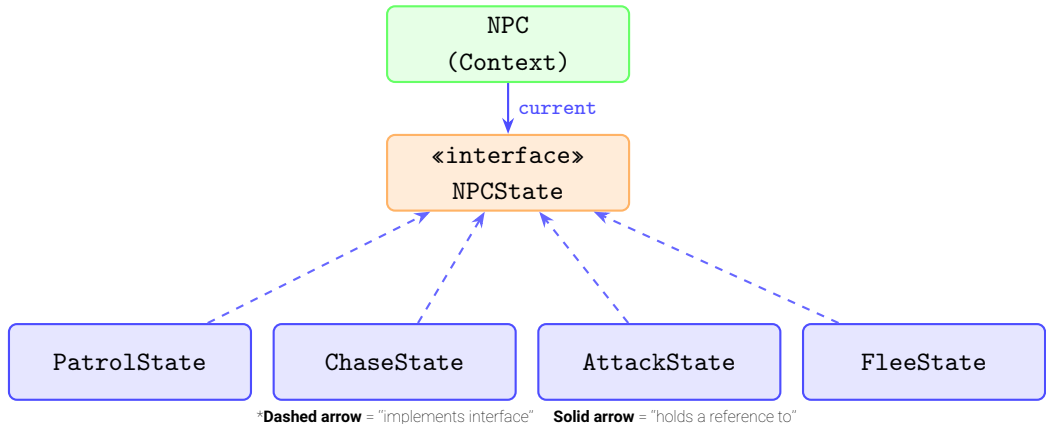
Enum with Per-State Behaviour

Rare Java pattern for completeness.

```
1  enum DoorState {
2      LOCKED {
3          @Override public void onEnter() {
4              System.out.println("*click* Door locked.");
5          }
6          @Override public DoorState transition(String event) {
7              return event.equals("unlock") ? UNLOCKED : this;
8          }
9      },
10     UNLOCKED { ... };
11
12     public abstract void onEnter();
13     public abstract DoorState transition(String event);
14 }
```

Each constant overrides behaviour directly. Compact, but harder to navigate with many states.

Approach B: State Pattern (GoF)



Context holds a reference to the current **NPCState**. All method calls are delegated to it.
Swapping behaviour = swapping the reference.

State Pattern: The Interface

How to read this interface:

```
1  public interface NPCState {  
2  
3      void onEnter(NPC npc);  
4  
5      void update(NPC npc);  
6  
7      void onExit(NPC npc);  
8  }
```

- **onEnter**: setup when state becomes active
- **update**: behaviour on each game tick
- **onExit**: cleanup before leaving the state

The context (**NPC**) calls these hooks, while state-specific logic stays inside each concrete state class.

Analogy

Think of a state like an actor: **onEnter** = walk on stage, **update** = perform every tick, **onExit** = exit and clean up props.

State Pattern: A Concrete State

```
1  public class PatrolState implements NPCState {
2      @Override
3      public void onEnter(NPC npc) {
4          npc.playAnimation("walk");
5          npc.setSpeed(1.5);
6          System.out.println(npc.getName() + " begins patrolling.");
7      }
8      @Override
9      public void update(NPC npc) {
10         npc.moveAlongWaypoints();
11         // Guard: player spotted -> transition to CHASE
12         if (npc.canSeePlayer()) {
13             npc.setState(new ChaseState());
14         }
15     }
16     @Override
17     public void onExit(NPC npc) {
18         // nothing to clean up
19     }
20 }
```

State Pattern: Why This Design Helps

Key insight

PatrolState owns the decision to transition. The context (**NPC**) contains **zero if-else** chains about behaviour.

*Note: we call **new ChaseState()** each time we transition. For most games this is fine. In performance-critical code you would cache state instances and reuse them.*

State Pattern: The Context (NPC)

What matters

```
1  public class NPC {  
2      private NPCState currentState = new PatrolState();  
3  
4      public NPC() {  
5          currentState.onEnter(this);  
6      }  
7  
8      public void update() {  
9          currentState.update(this); // delegate behavior  
10     }  
11  
12     public void setState(NPCState next) {  
13         currentState.onExit(this);  
14         currentState = next;  
15         currentState.onEnter(this);  
16     }  
17 }
```

- **Delegation:** NPC does not decide behavior; the active state does.
- **Single transition point:** `setState(...)` centralizes lifecycle calls.
- **Lifecycle contract:** every transition is `onExit` -> `onEnter`.
- **Scalable design:** no giant `if-else` in the context class.

Interactive Break #2 -- Enum vs. State Pattern

Scenario: you are building an RPG with the following NPC states:

IDLE, PATROL, ALERT, CHASE, COMBAT, STUNNED, FLEE, DEAD, RESPAWNING, CUTSCENE_LOCK, DIALOGUE, SCRIPTED_EVENT

1. Which approach would you choose and **why**?
2. A new "**BERSERK**" mode needs to be added in week 3 of development. How does adding it differ between Approach A and Approach B?
3. Two team members will work on NPC states simultaneously. Which approach supports parallel development better?

Discuss with your neighbour -- 5 minutes

There is no single correct answer. Be ready to argue your choice.

Interactive Break #2 -- Answer

State Pattern wins here

- 12 states → 12 separate, focused classes. Each is easy to read and test in isolation.
- Adding **BerserkState**: create one new file, implement the interface. **Zero changes** to existing code.
- Two devs can work on **StunnedState** and **DialogueState** without merge conflicts.

Rule of thumb: ≤ 5 states and no team collaboration → enum. 6+ states or team project → State Pattern.

Enum-based trade-offs

- Adding **BERSERK**: open the enum, add a constant, update every **switch** that handles states. High change surface.
- Parallel work: both devs touch the same file → merge conflicts.
- But: for 3-state prototypes, the enum is **much faster** to write. Know your context.

What Makes NPCs Feel Alive?

Classic game AI loop:

1. **Sense** the environment (see player? low HP? near patrol point?)
2. **Decide** what to do (check transition guards)
3. **Act** (move, attack, play animation)

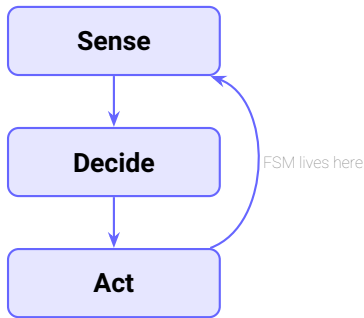
What Makes NPCs Feel Alive?

Classic game AI loop:

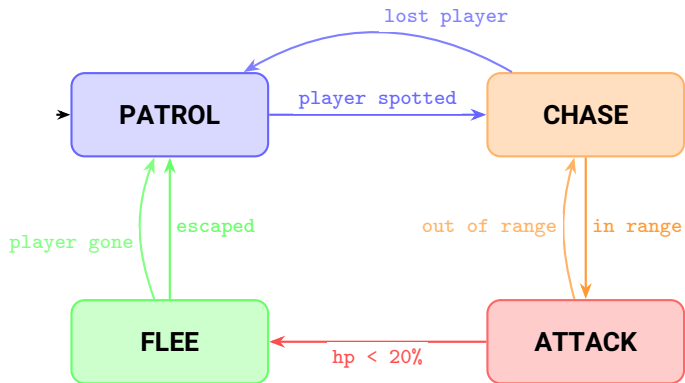
1. **Sense** the environment (see player? low HP? near patrol point?)
2. **Decide** what to do (check transition guards)
3. **Act** (move, attack, play animation)

A state machine handles step 2 elegantly:

- Current state defines the *current goal*
- Guards encode the *trigger conditions*
- **onEnter/onExit** handle *setup and cleanup*



The NPC State Machine



Implementing PATROL

```
1  public class PatrolState implements NPCState {
2      @Override
3      public void update(NPC npc) {
4          npc.moveAlongWaypoints();
5          if (npc.canSeePlayer()) {
6              npc.setState(new ChaseState()); // -> CHASE
7          }
8      }
9
10     ...
11 }
```

moveAlongWaypoints()

The NPC walks between a list of patrol waypoints in a loop. The exact path logic lives in the **NPC** class (or a helper); **PatrolState** just calls it.

CHASE as Transition Table

Idea: same State interface, but now we focus on decision logic only.

Current State	Guard (condition)	Next State
CHASE	<code>distanceToPlayer() < 2.0</code>	ATTACK
CHASE	<code>!canSeePlayer()</code>	PATROL
CHASE	otherwise (keep chasing)	CHASE

Runtime action while staying in CHASE

`moveToward(player)` on every tick.

Why State Flickering Happens (CHASE vs AT-TACK)

CHASE

```
1  void onEnter(NPC npc) {
2      npc.playAnimation("run");
3  }
4
5  void update(NPC npc) {
6      if (npc.distanceToPlayer() < 2.0) {
7          npc.setState(new AttackState());
8      }
9  }
```

ATTACK

```
1  void onEnter(NPC npc) {
2      npc.playAnimation("attack");
3  }
4
5  void update(NPC npc) {
6      if (npc.distanceToPlayer() >= 2.0) {
7          npc.setState(new ChaseState());
8      }
9  }
```

Boundary problem

Both transitions use the same boundary **2.0**. If distance oscillates around that value (e.g. 1.99, 2.01, 1.98, 2.00), the NPC can switch state almost every tick.

Fixing Flickering with Hysteresis

CHASE (enter ATTACK earlier)

```
1  void onEnter(NPC npc) {
2      npc.playAnimation("run");
3  }
4
5  void update(NPC npc) {
6      if (npc.distanceToPlayer() < 2.0) {
7          npc.setState(new AttackState());
8      }
9  }
```

ATTACK (leave ATTACK later)

```
1  void onEnter(NPC npc) {
2      npc.playAnimation("attack");
3  }
4
5  void update(NPC npc) {
6      if (npc.distanceToPlayer() > 2.5) {
7          npc.setState(new ChaseState());
8      }
9  }
```

Hysteresis gap

Use different thresholds for entering and leaving a state. Here the gap **2.0 .. 2.5** prevents rapid back-and-forth switching.

FLEE as Transition Table

FLEE has one key question: “am I safe yet?”

Current State	Guard (condition)	Next State
FLEE	<code>distanceToPlayer() > SAFE_DISTANCE</code>	PATROL
FLEE	<code>!canSeePlayer()</code>	PATROL
FLEE	otherwise (still threatened)	FLEE

Runtime action while staying in FLEE

`moveAwayFrom(player)` on every tick, usually with higher speed.

What Changes from State to State?

Usually stays the same

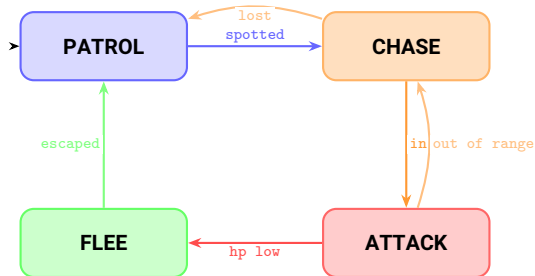
- Interface shape: **onEnter**, **update**, **onExit**
- Transition mechanism:
npc.setState(...)
- Tick-based execution via game loop

Actually changes

- **Action per tick** (move toward, attack, run away, ...)
- **Guards** (distance, visibility, HP threshold)
- **Entry/exit effects** (animation, speed, sound, logs)

Discussion: Extending the NPC FSM

Step 1 -- Start from the old 4-state model



We need to change the logic

Requirement change: add **IDLE** for “player far away” behavior. What **new state** and **new transitions** are needed?

Discussion: What Must Change?

New/changed behavior

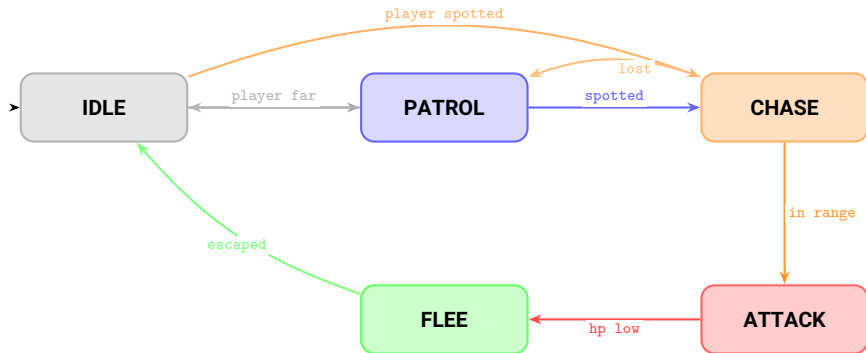
- Add new state: **IDLE**
- **PATROL** -> **IDLE** when player is far away
- **IDLE** -> **PATROL** after timer expires (5s)
- **IDLE** -> **CHASE** if player gets close/spotted
- (optional design choice) **FLEE** -> **IDLE** after escape

Key architecture question

Where does this change go?

- Add one new state class (**IdleState**)
- Add only relevant transitions in affected states
- Keep existing states mostly untouched

Updated Diagram + Resolution



Why this is a good design

- We extend behavior by adding one state and a few transitions.
- We avoid rewriting one large monolithic decision block.
- Changes stay local, readable, and easy to test.

Lecture Summary

1 -- FSM Concepts

- States, transitions, guards
- Initial state, events
- State diagrams (circles + arrows)
- Formal: $(\mathbf{S}, \mathbf{\Sigma}, \delta, \mathbf{s_0}, \lambda)$

3 -- Graphs

- FSM = directed graph
- Adjacency matrix vs. list
- Graph algorithms apply

2 -- Moore & Mealy

- Moore: output with *states*
- Mealy: output with *transitions*
- Being $\rightarrow$ Moore; Reacting $\rightarrow$ Mealy
- NPC AI = Moore machine

4 -- Java Implementations

- Enum-based: fast, good for small machines
- State Pattern: extensible, OCP-compliant
- Interface: **onEnter** / **update** / **onExit**